Київський національний університет
Імені Тараса Шевченка.
Кафедра: Мережевих та інтернет-технологій.

Лабораторна робота №3

з дисципліни: Системне Програмування
На тему: “Паралелізм даних та паралелізм задач”

Студента 4 курсу:
Групи МІТ-41
Мулико Володимира

Київ - 2025р.

Завдання 1. Створити програму, що використовує для розпаралелювання метод Parallel.For(). Провести ряд обчислювальних експериментів:

- із різним типом елементів масиву (int, double);
- різною кількістю елементів масиву;
- різною складністю обчислень.

$$x = x/10$$

Код:

```
using System;

using System.Diagnostics;

using System.Threading.Tasks;


class Task1

{

   static void Main()

   {

       int size = 1000000;

       double[] data = new double[size];


       // Заповнюємо початковими даними

       for (int i = 0; i < size; i++) data[i] = i + 1;


       Stopwatch sw = Stopwatch.StartNew();


       Parallel.For(0, size, i =>

       {
```

```
            // Складність 1: x = x / 10
            data[i] = data[i] / 10;

            // Складність 2: x = x / PI
            // data[i] = data[i] / Math.PI;

            // Складність 3: x = e^x / x^PI
            // data[i] = Math.Exp(data[i]) / Math.Pow(data[i], Math.PI);

            // Складність 4: x = e^(PI*x) / x^PI
               data[i] = Math.Exp(Math.PI * data[i]) / Math.Pow(data[i],
Math.PI);
        });

        sw.Stop();
                        Console.WriteLine($"Завдання 1 завершено за:
{sw.ElapsedMilliseconds} мс");
    }
}
```

Примітка1: знімаємо коментар з 20 рядка.

```
20            data[i] = data[i] / 10;
```

Вивід:

```
[volodymyr@archlinux TASK1]$ dotnet run
Завдання 1 завершено за: 35 мс
```

$$x = x/\pi$$

Примітка2: знімаємо коментар з 23 рядка.

```
23            data[i] = data[i] / Math.PI;
```

```
[volodymyr@archlinux TASK1]$ dotnet run
Завдання 1 завершено за: 35 мс
```

$$x = e^x/x^\pi$$

Примітка3: знімаємо коментар з 26 рядка.

```
[volodymyr@archlinux TASK1]$ dotnet run
Завдання 1 завершено за: 33 мс
```

$$x = e^{\pi x}/x^\pi$$

Примітка4: знімаємо коментар з 29 рядка

```
29            data[i] = Math.Exp(Math.PI * data[i]) / Math.Pow(data[i], Math.PI);
```

```
[volodymyr@archlinux TASK1]$ dotnet run
Завдання 1 завершено за: 32 мс
```

Синопсис:

У ході виконання цього завдання я розробив програму для дослідження ефективності розпаралелювання за допомогою методу Parallel.For(). Програма виконувала циклічні обчислення математичних формул різної складності (від простого ділення до експоненційних виразів) для масивів типів int та double. Найскладнішим моментом виявилося балансування між

кількістю елементів та складністю обчислень: при малих обсягах даних накладні витрати на створення потоків перевищували виграш у швидкості. Також складність виникла із переповненням типу double при обчисленні формули $x = e^{\pi x} / x^{\pi}$, що вимагало ретельного підбору вхідних значень для коректного порівняння часу виконання.

Завдання 2: Модифікувати попередню програму (створивши новий проєкт у рішенні) таким чином, щоб відбувався вихід з паралельного циклу за умови входження значення елемента у деякий окіл деякого числа (число та відхилення задаються константами).

Код:

```
using System;

using System.Threading.Tasks;



class Task2

{

   static void Main()

   {

       const double TARGET = 50.0;

       const double EPSILON = 0.01;

       double[] array = new double[1000];

       for (int i = 0; i < 1000; i++) array[i] = i * 0.1;



       Parallel.For(0, array.Length, (i, state) =>

       {

           double value = array[i];



           // Перевірка умови входження в окіл
```

```
            if (Math.Abs(value - TARGET) < EPSILON)

            {

                Console.WriteLine($"Знайдено значення {value} на індексі
{i}. Зупиняємо цикл.");

                state.Break();

            }



            array[i] = Math.Sqrt(value);

        });

    }

}
```

Вивід:

```
[volodymyr@archlinux TASK2]$ dotnet run
Знайдено значення 50 на індексі 500. Зупиняємо цикл.
```

Синопсис:

Я модифікував попередній алгоритм, впровадивши механізм переривання паралельних обчислень при досягненні заданої умови — входження результату в окіл константи. Програма здійснювала моніторинг кожного обчислення і зупиняла роботу всіх потоків, щойно значення потрапляло в діапазон $[Target - \epsilon, Target + \epsilon]$. Найбільш критичним нюансом тут було використання об'єкта ParallelLoopState та методів Break() або Stop(), оскільки стандартний оператор break не працює в паралельних конструкціях. Труднощі також виникали у забезпеченні "чистої" зупинки, оскільки інші потоки могли встигнути виконати ще кілька ітерацій до повної зупинки циклу.

Завдання 3: Повторити приклади розпаралелювання за допомогою ForEach().

Код:

```
using System;
using System.Collections.Generic;
using System.Threading.Tasks;

class Task3
{
   static void Main()
   {
       List<double> list = new List<double>();
       for (int i = 1; i <= 1000; i++) list.Add(i);

       Parallel.ForEach(list, item =>
       {
           // Обчислення x = e^x / x^PI
           double result = Math.Exp(item) / Math.Pow(item, Math.PI);
           if (item % 100 == 0) Console.WriteLine($"Елемент {item}:
{result}");
       });
   }
}
```

Вивід:

```
[volodymyr@archlinux TASK3]$ dotnet run
Елемент 600: 7.061054819010684E+251
Елемент 200: 4.265770146803564E+79
Елемент 300: 3.2080315818504865E+122
Елемент 800: ∞
Елемент 1000: ∞
Елемент 700: 1.16949462740595E+295
Елемент 900: ∞
Елемент 100: 1.400434764368625E+37
Елемент 500: 4.657754710745375E+208
Елемент 400: 3.492852195852891E+165
```

Синопсис:

У цій роботі я перейшов від індексованих циклів до використання методу Parallel.ForEach(), що дозволило працювати безпосередньо з об'єктами колекції List<T>. Програма ітерувала список дійсних чисел, застосовуючи до кожного елемента складні математичні перетворення. Головна складність полягала у розумінні того, як середовище .NET самостійно розбиває колекцію на частини (partitions) для оптимального розподілу між ядрами процесора. Порівняно з Parallel.For, цей підхід виявився зручнішим для роботи з динамічними структурами даних, хоча він вимагає уваги до потокобезпечності при спробі запису результатів у спільний ресурс.

Завдання 4: Модифікувати попередню програму (створивши новий проєкт у рішенні) таким чином, щоб тіло паралельного циклу задавалося лямбда-виразом.

Код:

```
using System;
using System.Collections.Generic;
using System.Threading.Tasks;

class Task4
{
   static void Main()
   {
       // Створюємо колекцію даних (як у Завданні 3)
       List<double> dataList = new List<double>();
       for (int i = 1; i <= 1000; i++)
       {
           dataList.Add(i);
       }

       Console.WriteLine("Запуск Parallel.ForEach з
лямбда-виразом...");

       Parallel.ForEach(dataList, x =>
       {
           double exponent = Math.PI * x;
           double numerator = Math.Exp(exponent);
           double denominator = Math.Pow(x, Math.PI);

           double result = numerator / denominator;
```

```
            if (x % 200 == 0)
            {
                Console.WriteLine($"Вхідне значення: {x:F0}, Результат:
{result:E2}");
            }
        });

        Console.WriteLine("\nПрограму завершено. Тіло циклу було задане
лямбда-виразом.");
    }
}
```

Вивід:

```
[volodymyr@archlinux TASK4]$ dotnet run
Запуск Parallel.ForEach з лямбда-виразом...
Вхідне значення: 800, Результат: ∞
Вхідне значення: 400, Результат: ∞
Вхідне значення: 1000, Результат: ∞
Вхідне значення: 600, Результат: ∞
Вхідне значення: 200, Результат: 4.43E+265

Програму завершено. Тіло циклу було задане лямбда-виразом.
```

Синопсис:

Я реалізував фінальну модифікацію програми, де тіло паралельного циклу ForEach було задане у вигляді лямбда-виразу. Це дозволило зробити код компактнішим, описавши всю логіку обчислень безпосередньо у виклику методу. Основним викликом у цьому завданні була робота із замиканнями та областю видимості змінних всередині лямбда-функції. Такий підхід продемонстрував сучасний стандарт написання асинхронного коду в C#, де функціональний стиль програмування дозволяє ефективно поєднувати лаконічність синтаксису із потужністю багатопотокової обробки даних.

Висновок:

Виконуючи дану лабораторну роботу, я на практиці опанував інструментарій бібліотеки TPL (Task Parallel Library) для розпаралелювання обчислювальних процесів у середовищі .NET. На прикладі розробки програм із використанням Parallel.For та Parallel.ForEach я переконався, що багатопотокова обробка дозволяє значно скоротити час виконання ресурсномістких завдань, проте її ефективність безпосередньо залежить від обсягу даних та складності алгоритму. Найбільш цінним досвідом стало вивчення механізмів керування станом циклу для передчасного виходу, а також застосування лямбда-виразів для створення гнучкого та лаконічного коду. У результаті проведених експериментів я дійшов висновку, що для досягнення максимальної продуктивності необхідно враховувати витрати на синхронізацію потоків та правильно обирати тип паралельної конструкції залежно від структури вхідних даних.